Homework Set 2– Practical Skills and Concurrency

Due Date: Thursday, Apr 25, 2026

1. Regular expressions and shell programming

Tsinghua has a very complicated email domain naming convention. There are two campus-wide email domains:

1) Faculty and regular staff members use the @mail.tsinghua.edu.cn domain, and it can also be written as @tsinghua.edu.cn.
2) Students use the @mails.tsinghua.edu.cn domain.

Also, departments and large labs have their own email servers. For example, our institute has the @iiis.tsinghua.edu.cn domain.
Tsinghua alumni can apply for email addresses in the @mail.tsinghua.org.cn domain.

Consider that you have a text file containing email addresses with both Tsinghua email addresses and non-Tsinghua addresses, one address per line. Please complete the following tasks:

1) Write a regular expression that matches:
   a) All student email addresses ONLY.
   b) All faculty and regular staff member email addresses ONLY.
   c) All campus-wide email addresses.
2) Write a regular expression that matches all Tsinghua email addresses, including alumni addresses.
3) Write a regular expression that extracts all department server names, not including campus-wide servers such as mail, mails, or tsinghua. For example, in the email address xuw@iiis.tsinghua.edu.cn, the department server name is iiis. You should look up regular expression documentation to learn how to extract capturing groups.
4) Assume that you store the matching server names from question 3) in a text file called departments.txt, one name per line. Write a single-line bash command that outputs the distinct department server names in **case-insensitive** alphabetical order, printing all names **in lowercase**, one per line. The command should also output the total number of such distinct department server names on a separate line after the list.

2. Basic synchronization

Threads A and B both share references to the same already initialized doubly linked List object, which currently contains multiple nodes (more than 2 nodes). Each thread calls the removeTail() method once. Consider the three scenarios (see the comments in the code) where thread A runs initially and then switches to thread B, which completes the entire removeTail() method before switching back to thread A. Which of these scenarios would lead to incorrect behavior of removeTail()? Incorrect behavior means that the final List object has missing deletions (a node that should have been removed is not removed), incorrect pointers, a broken list structure, double free, or any other invalid memory behavior.

class Node{

```
    Object entry;
    Node next;
    Node prev;
    Node(Object entry){
        this.entry=entry;
    }
}
class List{
    Node head = new Node(null);
    Node tail = head;
    /*
    // initialization operations
    */
    void removeTail(){
        Node oldTail = tail; //scenario 1: A switches to B after this line
        tail = tail.prev; //scenario 2: A switches to B after this line
        tail.next = null; //scenario 3: A switches to B after this line
        delete oldTail;
    }
}
```

3. Monitors

Consider the following code that implements a synchronized unbounded queue using monitors that we discussed in lecture:

```
1. Lock lock;
2. Condition dataready;
3. Queue queue;
4. AddToQueue(item) {
5.   lock.Acquire(); // Get Lock
6.   queue.enqueue(item); // Add item
7.   dataready.signal(); // Signal any waiters
8.   lock.Release(); // Release Lock
9. }
10. RemoveFromQueue() {
11.   lock.Acquire(); // Get Lock
12.   while (queue.isEmpty()) {
13.     dataready.wait(&lock); // If nothing, sleep
14.   }
15.   item = queue.dequeue(); // Get next item
16.   lock.Release(); // Release Lock
17.   return(item);
18. }
```

1) Assume that we have multiple producers running `AddToQueue()` and multiple consumers running `RemoveFromQueue()`. Do you need to make any changes to the code? If yes, specify the changes in the above code by indicating the line you need to modify, the line #'s between which you need to add new code, or the line # you need to delete. If not, use no more than two sentences to explain why.

2) Change the code to implement a bounded queue, i.e., make sure that the producer cannot

write when the queue is full. Add your changes in the empty space of the code below.

3) Implement a new function, `ReadFromQueue()`, which uses the function "`item = queue.read()`" to read an item from the queue without removing it.

4. New locking primitives using semaphores

As we have discussed in class, a *Reentrant Lock* is owned by the thread last successfully locking, but not yet unlocking it. A thread invoking RLock() will return, successfully acquiring the lock, when the lock is *not* owned by another thread. The RLock() method will also return immediately if the current thread already owns the lock (the unlock only need to happen once, no matter how many times a thread locks it). Otherwise the thread is put on sleep until the lock holder calls RUnLock().

Please use the Semaphore P() and V() primitive to implement an reentrant lock. You can obtain the thread id by calling the function `getMyTID()`. E.g. `int cur_id = getMyTID();`

```
// define globle variables here
…
void RLock() {
  // add your code here
}
void RUnLock() {
  // add your code here
}
```

5. Modeling the real world

A machine learning inference server has one GPU worker and a waiting queue with N slots for pending requests. The GPU worker goes to sleep if there are no requests to process. If a client request arrives and the GPU is currently busy while all N waiting slots are occupied, the request is dropped immediately. Otherwise, the request joins the waiting queue and waits. If the GPU worker is asleep when a request arrives, the request wakes it up. Whenever the GPU worker becomes available, it selects the next request from the waiting queue with the shortest estimated processing time and processes it.

1) The following code segment models the GPU worker and each client request as independent, concurrent threads. Fill in the following blanks to ensure that the GPU worker and the client requests are synchronized and deadlock-free. Assume each semaphore has P() and V() functions available as semaphore.P() and semaphore.V().

Semaphore gpuReady = 0;
Semaphore accessQueue = 1;

```
Semaphore requestReady = 0;
int numberOfFreeQueueSlots = N;

void GPUWorker() {
      while (true) {
            _____________________________________
            _____________________________________

            numberOfFreeQueueSlots += 1;
            _____________________________________

            processRequest(); // Process one request on the GPU
            _____________________________________
      }
}

void ClientRequest() {
      _____________________________________
      if (numberOfFreeQueueSlots > 0) {
            numberOfFreeQueueSlots -= 1;
            _____________________________________
            _____________________________________
            _____________________________________
            waitForCompletion(); // Client waits until its request is processed
      } else {
            _____________________________________
            dropRequest(); // No available queue slot
      }
}
```

2) Which of the semaphores are used for mutex, and which are used for scheduling constraints, respectively?

3) Describe a scenario where a request can get starved, i.e., remain in the waiting queue forever without being processed. Is there a deadlock when this happens? Can starvation still happen if the GPU worker randomly picks one request from the waiting queue? Can you propose a way to guarantee that no starvation happens?